

Image Classification using pre-trained CNNs and Deployment with Streamlet

Step (01): Data selection

- In this step, we used the `tf_flowers` dataset from TensorFlow Datasets, which contains images of five flower categories: *dandelion*, *daisy*, *tulips*, *sunflowers*, and *roses*.
- We loaded the dataset and split it into training (85%) and validation (15%) subsets using the `tfds.load()` function.

Step (02): Model Implementation (Transfer Learning) A. **Data Preparation**

- In this step, we resized all images to 224×224 pixels to match the input requirements of pre-trained CNNs like MobileNetV2 and ResNet50.
- We applied data augmentation techniques such as random flipping, rotation, and zoom to improve generalization.
- Images were also normalized (scaled from 0–255 to 0–1) and organized into batches of 32 for efficient training.
- We prepared separate pipelines for training (with augmentation) and validation (without augmentation) using TensorFlow's `tf.data` API.

B. Pre-trained CNN Model

- Convolutional Neural Networks (CNNs) automatically learn to extract spatial features from images — such as edges, textures, and object parts — making them highly effective for image classification tasks.
- In this step, we built two image classification models using pre-trained convolutional neural networks (CNNs) — MobileNetV2 and ResNet50 — as the base.
- These models were originally trained on the large ImageNet dataset, so they already know how to detect useful visual patterns like edges, textures, and object shapes.

To adapt them for our 5-class flower classification task, we did the following:

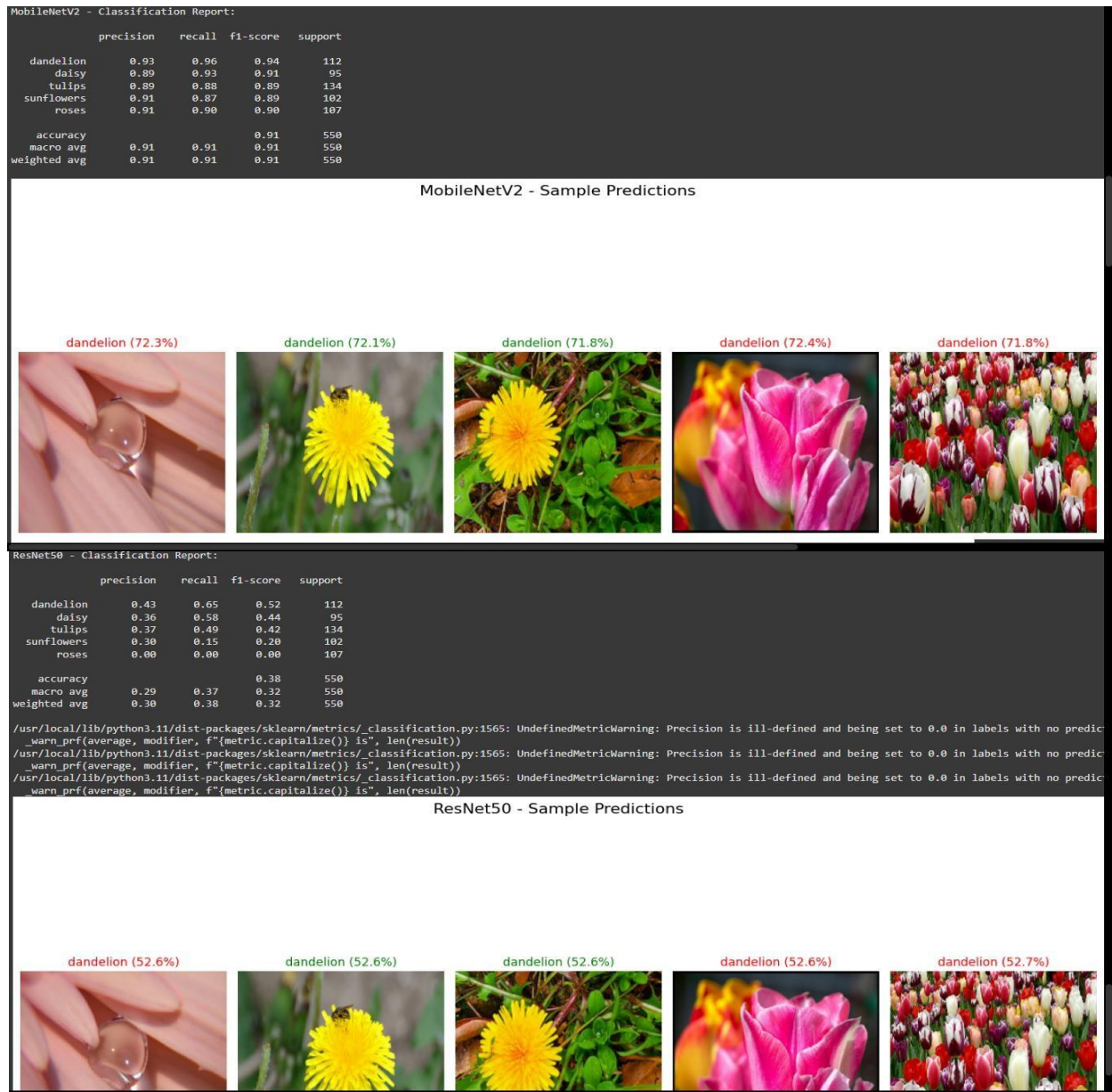
1. **Loaded the pre-trained models without their top (classification) layers** by setting `include_top=False`.
2. **Added a custom classification head** on top of each model:
 - GlobalAveragePooling2D: reduces spatial dimensions
 - Dense layer (128 units): learns task-specific features
 - Dropout layer (0.3): helps prevent overfitting
 - Final Dense layer (5 units with softmax): outputs class probabilities
3. **Froze the base model layers** (`base_model.trainable = False`) so that during initial training, only the new classification layers are updated. This ensures we don't destroy the valuable pre-learned features.

This setup allows us to train efficiently using a smaller dataset (like `tf_flowers`), while still benefiting from the deep knowledge encoded in powerful CNN architectures.

C. Evaluation

We evaluate model performance by:

1. **Plotting accuracy and loss curves** – to see how well the model trained and whether it overfit or underfit.
2. **Confusion matrix and classification report** – to assess prediction quality across all classes.
3. **Visualizing sample predictions** – to check if the model predicts correctly and confidently on real images.



Step (03): Web App Deployment Using Stream lit

This Streamlit web app is a flower image classifier that uses deep learning models (MobileNetV2 and ResNet50) trained on the TF Flowers dataset. Users can upload a .jpg or .png image of a flower, and the app processes the image, predicts the flower type, and displays the top predicted class with its confidence score. It also shows the top 3 class probabilities for better insight. The app provides visual feedback by displaying the uploaded image along with the predictions in a user-friendly interface.